Structure of Programming Language

Using Python

Conditional Statement

Objectives

At the end of the module the student is expected.

1. **List and Explain the different conditional commands in Python**

2. **Identify the syntax of the conditional commands used in python.**

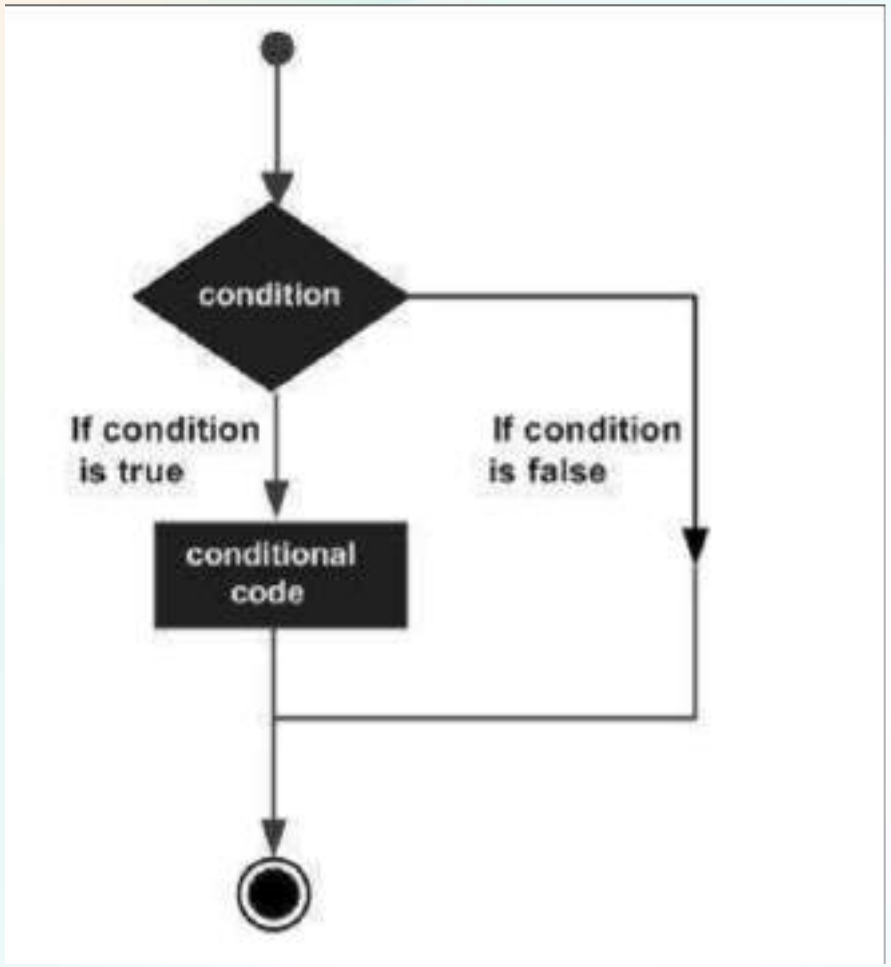
3. **Demonstrate how to use differnt conditional commands in Python**

What are Conditional Statements?

Conditional Statement in Python perform different computations or actions depending on whether a specific Boolean constraint evaluates to true or false. Conditional statements are handled by IF statements in Python.

Decision-making is the anticipation of conditions occurring during the execution of a program and specified actions taken according to the conditions. Decision structures evaluate multiple expressions, which produce TRUE or FALSE as the outcome. You need to determine which action to take and which statements to execute if the outcome is TRUE or FALSE otherwise.

Decision making structure



Python programming language assumes any non-zero and non-null values as TRUE, and any zero or null values as FALSE value.

Types of decision-making statements.

Statement	Description
if statements	An if statement consists of a Boolean expression followed by one or more statements.
if...else statements	An if statement can be followed by an optional else statement, which executes when the boolean expression is FALSE.
nested if statements	You can use one if or else if statement inside another if or else if statement(s).

IF Statement

The IF statement is similar to that of other languages.

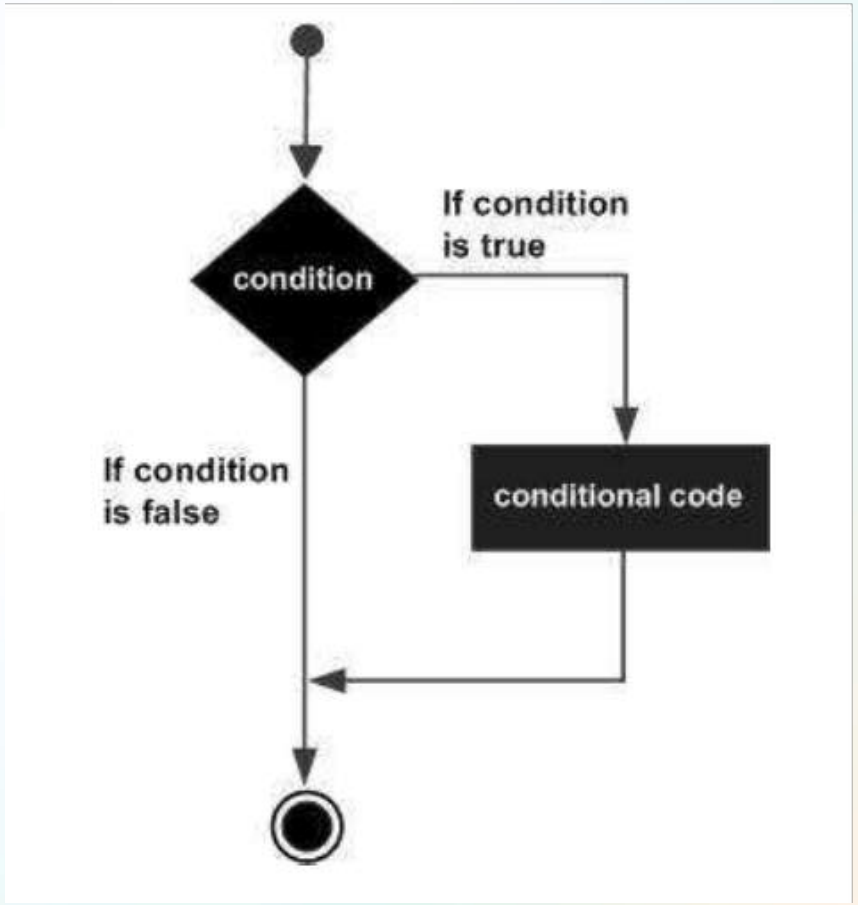
The if statement contains a logical expression using which the data is compared and a decision is made based on the result of the comparison.

Syntax / Flow Diagram

```
if expression:
    statement(s)
```

If the boolean expression evaluates to TRUE, then the block of statement(s) inside the if statement is executed.

In Python, statements in a block are uniformly indented after the : symbol. If boolean expression evaluates to FALSE, then the first set of code after the end of block is executed.



Example

```python
# If the number is positive, we print an appropriate message
num = 3
if num > 0:
    print(num, "is a positive number.")
print("This is always printed.")

num = -1
if num > 0:
    print(num, "is a positive number.")
print("This is also always printed.")
```

Python Code

```
3 is a positive number.
This is always printed.
This is also always printed.
```

Program output

Explanation

In the above example, num > 0 is the test expression.

The body of if is executed only if this evaluates to True.

When the variable num is equal to 3, test expression is true and statements inside the body of if are executed.

If the variable num is equal to -1, test expression is false and statements inside the body of if are skipped.

The print() statement falls outside of the if block (unindented). Hence, it is executed regardless of the test expression.

Example 2

```
var1 = 100
if var1:
    print ("1 - Got a true expression value")
    print (var1)

var2 = 0
if var2:
    print ("2 - Got a true expression value")
    print (var2)
print ("Good bye!")
```

Program Code

```
1 - Got a true expression value
100
Good bye!
```

Program Output

IF...ELIF...ELSE Statements

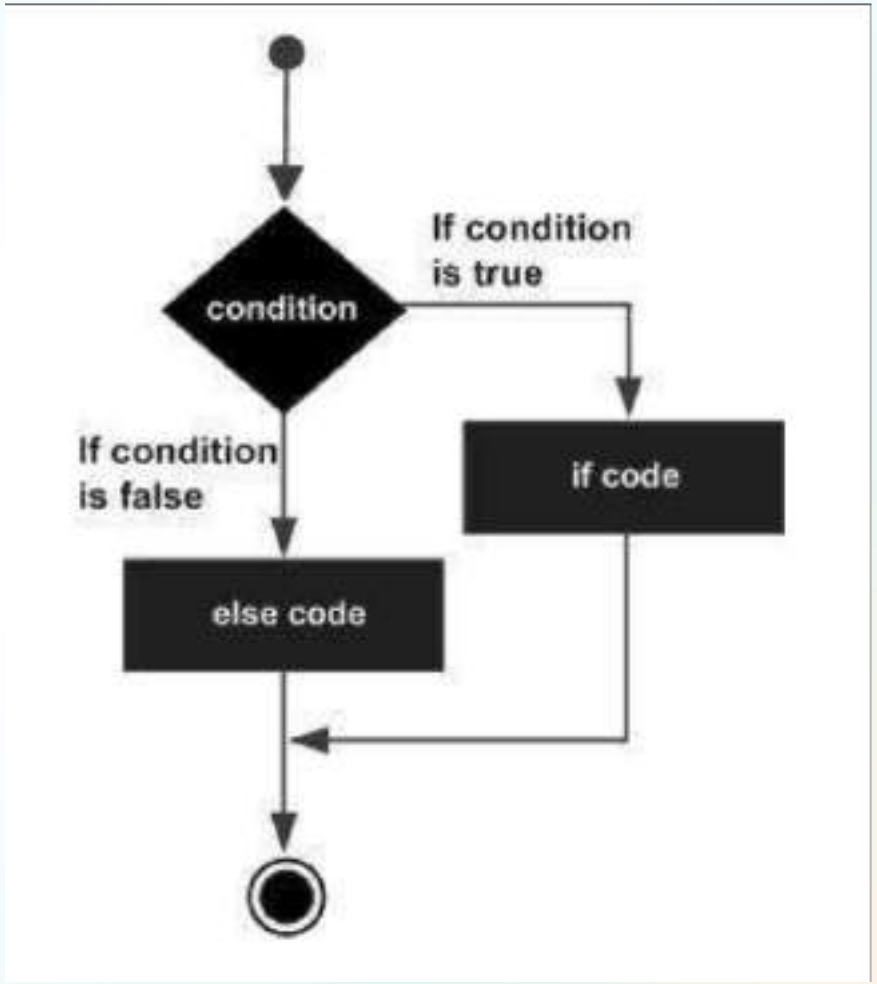
An else statement can be combined with an if statement.

An else statement contains a block of code that executes if the conditional expression in the if statement resolves to 0 or a FALSE value.

The else statement is an optional statement and there could be at the most only one else statement following if.

Syntax / Flow Diagram

```
if expression:
    statement(s)
else:
    statement(s)
```



The if..else statement evaluates test expression and will execute the body of if only when the test condition is True.

If the condition is False, the body of else is executed. Indentation is used to separate the blocks.

Example

```python
# Program checks if the number is positive or negative
# And displays an appropriate message
num = 3
# Try these two variations as well.
# num = -5
# num = 0
if num >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

Program Code

```
Positive or Zero
```

Program Output

Explanation

In the above example, when num is equal to 3, the test expression is true and the body of if is executed and the body of else is skipped.

If num is equal to -5, the test expression is false and the body of else is executed and the body of if is skipped.

If num is equal to 0, the test expression is true and body of if is executed and body of else is skipped.

Example 2

```
#Program to compute the discount
amount=int(input("Enter amount: "))
if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
else:
    discount=amount*0.10
    print ("Discount",discount)

print ("Net payable:",amount-discount)
```

Program Code

```
Enter amount: 600
Discount 30.0
Net payable: 570.0
```

Program Output

```
#Program to compute the discount
amount=int(input("Enter amount: "))
if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
else:
    discount=amount*0.10
    print ("Discount",discount)

print ("Net payable:",amount-discount)
```

Program Code

```
Enter amount: 1200
Discount 120.0
Net payable: 1080.0
```

Program Output

In the above example, discount is calculated on the input amount. Rate of discount is 5%, if the amount is less than 1000, and 10% if it is above 10000.

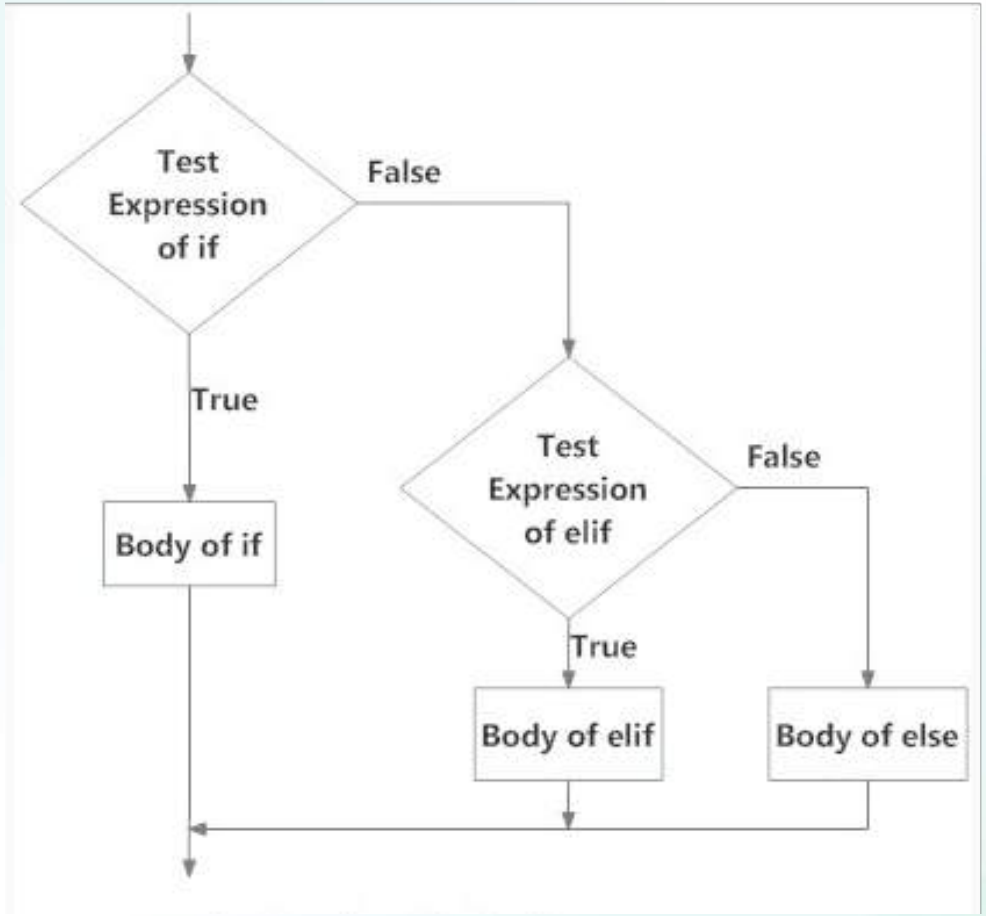
The elif Statement

The elif statement allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE.

Similar to the else, the elif statement is optional. However, unlike else, for which there can be at the most one statement, there can be an arbitrary number of elif statements following an if.

Syntax / Flow Diagram

```
if expression1:
    statement(s)
elif expression2:
    statement(s)
elif expression3:
    statement(s)
else:
    statement(s)
```



The elif is short for else if. It allows us to check for multiple expressions.

If the condition for if is False, it checks the condition of the next elif block and so on.

If all the conditions are False, the body of else is executed.

Only one block among the several if...elif...else blocks is executed according to the condition.

The if block can have only one else block. But it can have multiple elif blocks.

Example

```python
'''In this program,
we check if the number is positive or
negative or zero and
display an appropriate message'''
num = 3.4
# Try these two variations as well:
# num = 0
# num = -4.5
if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
```

Program Code

```
Positive number
```

Program Output

When variable num is positive, Positive number is printed.

If num is equal to 0, Zero is printed.

If num is negative, Negative number is printed.

Example 2

```python
amount=int(input("Enter amount: "))
if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
elif amount<5000:
    discount=amount*0.10
    print ("Discount",discount)
else:
    discount=amount*0.15
    print ("Discount",discount)
print ("Net payable:",amount-discount)
```

Program Code

```
Enter amount: 600
Discount 30.0
Net payable: 570.0
```

Program Output

```python
amount=int(input("Enter amount: "))
if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
elif amount<5000:
    discount=amount*0.10
    print ("Discount",discount)
else:
    discount=amount*0.15
    print ("Discount",discount)
print ("Net payable:",amount-discount)
```

Program Code

```
Enter amount: 3000
Discount 300.0
Net payable: 2700.0
```

Program Output

```python
amount=int(input("Enter amount: "))
if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
elif amount<5000:
    discount=amount*0.10
    print ("Discount",discount)
else:
    discount=amount*0.15
    print ("Discount",discount)
print ("Net payable:",amount-discount)
```

Program Code

```
Enter amount: 6000
Discount 900.0
Net payable: 5100.0
```

Program Output

Nested IF Statements

There may be a situation when you want to check for another condition after a condition
resolves to true. In such a situation, you can use the nested if construct.

In a nested if construct, you can have an if...elif...else construct inside another if...elif...else construct.

Syntax

```
if expression1:
    statement(s)
    if expression2:
        statement(s)
    elif expression3:
        statement(s)
    else
        statement(s)
elif expression4:
    statement(s)
else:
    statement(s)
```

We can have a if...elif...else statement inside another if...elif...else statement. This is called nesting in computer programming.

Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting. They can get confusing, so they must be avoided unless necessary.

Example

```python
'''In this program, we input a number
check if the number is positive or
negative or zero and display
an appropriate message
This time we use nested if statement'''

num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

Program Code

```
Enter a number: 5
Positive number
```

Program Output

```python
'''In this program, we input a number
check if the number is positive or
negative or zero and display
an appropriate message
This time we use nested if statement'''

num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

Program Code

```
Enter a number: -1
Negative number
```

Program Output

```python
'''In this program, we input a number
check if the number is positive or
negative or zero and display
an appropriate message
This time we use nested if statement'''

num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

Program Code

```
Enter a number: 0
Zero
```

Program Output

Example 2

```
num=int(input("enter number: "))
if num%2==0:
    if num%3==0:
        print ("Divisible by 3 and 2")
    else:
        print ("divisible by 2 not divisible by 3")
else:
    if num%3==0:
        print ("divisible by 3 not divisible by 2")
    else:
        print ("not Divisible by 2 not divisible by 3")
```

Program Code

```
enter number: 8
divisible by 2 not divisible by 3
```

Program Output

```
num=int(input("enter number: "))
if num%2==0:
    if num%3==0:
        print ("Divisible by 3 and 2")
    else:
        print ("divisible by 2 not divisible by 3")
else:
    if num%3==0:
        print ("divisible by 3 not divisible by 2")
    else:
        print ("not Divisible by 2 not divisible by 3")
```

Program Code

```
enter number: 15
divisible by 3 not divisible by 2
```

Program Output

```
num=int(input("enter number: "))
if num%2==0:
    if num%3==0:
        print ("Divisible by 3 and 2")
    else:
        print ("divisible by 2 not divisible by 3")
else:
    if num%3==0:
        print ("divisible by 3 not divisible by 2")
    else:
        print ("not Divisible by 2 not divisible by 3")
```

Program Code

```
enter number: 12
Divisible by 3 and 2
```

Program Output

```
num=int(input("enter number: "))
if num%2==0:
    if num%3==0:
        print ("Divisible by 3 and 2")
    else:
        print ("divisible by 2 not divisible by 3")
else:
    if num%3==0:
        print ("divisible by 3 not divisible by 2")
    else:
        print ("not Divisible by 2 not divisible by 3")
```

Program Code

```
enter number: 5
not Divisible by 2 not divisible by 3
```

Program Output

Single Statement Suites

If the suite of an if clause consists only of a single line, it may go on the same line as the header statement.

```
var = 100
if ( var == 100 ) : print ("Value of expression is 100")
print ("Good bye!")
```

```
Value of expression is 100
Good bye!
```

Loops in Python

Objectives

At the end of the module the student is expected to:

Define Python Loop

Discuss the different types of loop available in Python

- For Loop
- While Loop
- Break and continue

List and explain the Loop Control Statements

- break statement
- continue statement
- pass statement

Topics

Loops

- While loop
- for loop
- Nested loops

Loop Control Statements

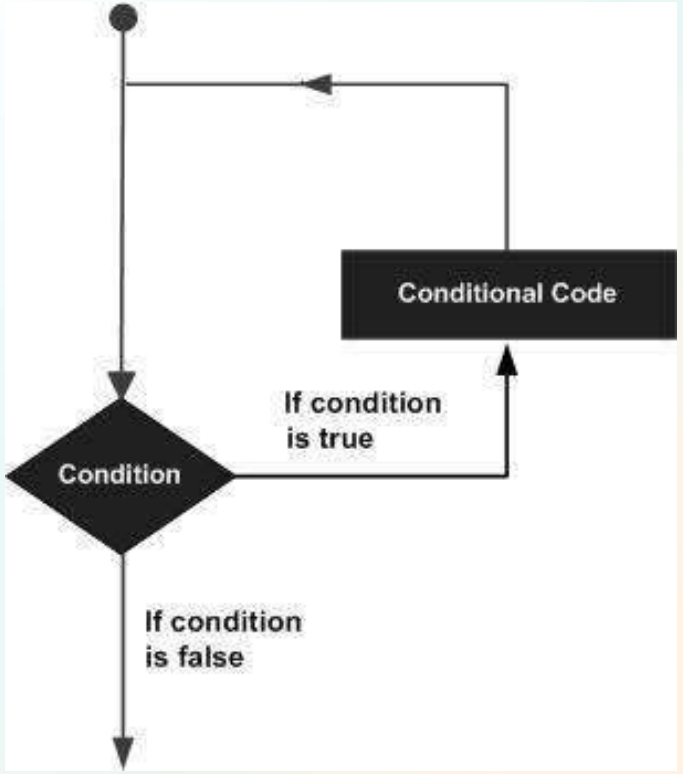
- break statement
- continue statement
- pass statement

What is Python loop

In general, statements are executed sequentially- The first statement in a function is executed first, followed by the second, and so on. There may be a situation when you need to execute a block of code several number of times.

Programming languages provide various control structures that allow more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times.



Types of loops

Loop Type	Description
while loop	Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
for loop	Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
nested loops	You can use one or more loop inside any another while, or for loop.

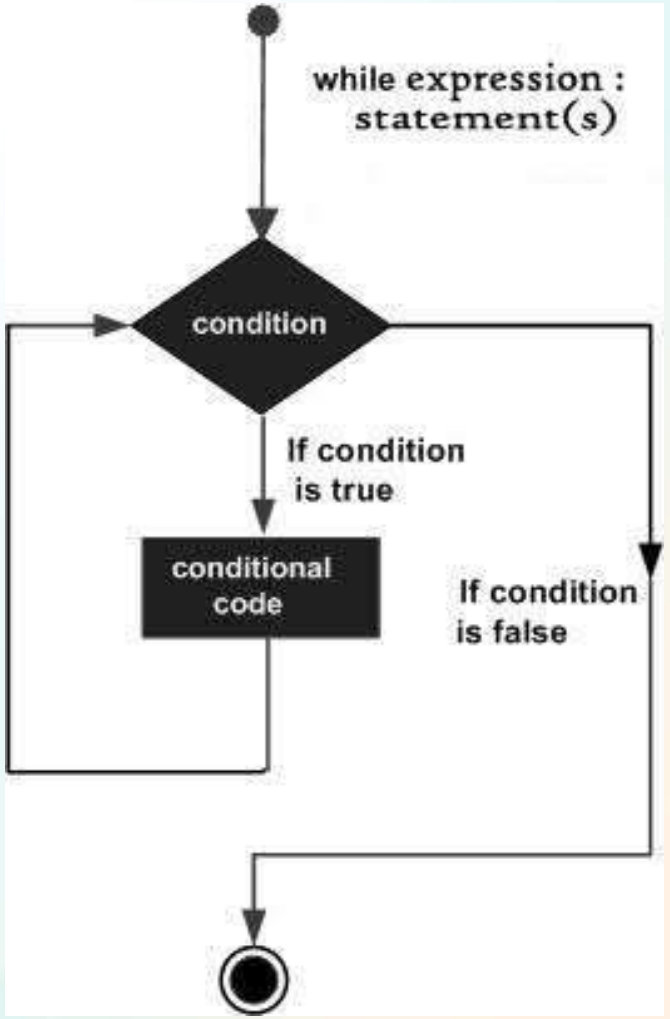
What is while loop in Python?

The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true.

We generally use this loop when we don't know the number of times to iterate beforehand.

Syntax and Flowchart of while Loop

```
while test_expression:
    Body of while
```



In the while loop, test expression is checked first. The body of the loop is entered only if the test_expression evaluates to True. After one iteration, the test expression is checked again.

This process continues until the test_expression evaluates to False.
In Python, the body of the while loop is determined through indentation.

The body starts with indentation and the first unindented line marks the end.

Python interprets any non-zero value as True. None and 0 are interpreted as False.

Example: Python while Loop

```python
# Program to add natural
# numbers up to
# sum = 1+2+3+...+n

# To take input from the user,
# n = int(input("Enter n: "))
n = 10
# initialize sum and counter
sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i+1    # update counter

# print the sum
print("The sum is", sum)
```

Program Code

```
The sum is 55
```

Program Output

In the above program, the test expression will be True as long as our counter variable i is less than or equal to n (10 in our program).

We need to increase the value of the counter variable in the body of the loop. This is very important (and mostly forgotten). Failing to do so will result in an infinite loop (never-ending loop).

Finally, the result is displayed.

Example 2

```
count = 0
while (count < 9):
    print ('The count is:', count)
    count = count + 1
print ("Goodbye!")
```

Program Code

```
The count is: 0
The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5
The count is: 6
The count is: 7
The count is: 8
Goodbye!
```

Program Output

The block here, consisting of the print and increment statements, is executed repeatedly until count is no longer less than 9. With each iteration, the current value of the index count is displayed and then increased by 1.

Using else Statement with Loops

Python supports having an else statement associated with a loop statement.

- If the else statement is used with a for loop, the else statement is executed when the loop has exhausted iterating the list.
- If the else statement is used with a while loop, the else statement is executed when the condition becomes false.

Example While loop with else

```
count = 0
while count < 5:
    print (count, " is less than 5")
    count = count + 1
else:
    print (count, " is not less than 5")
```

Program Code

```
0  is less than 5
1  is less than 5
2  is less than 5
3  is less than 5
4  is less than 5
5  is not less than 5
```

Program Output

The example illustrates the combination of an else statement with a while statement that prints a number as long as it is less than 5, otherwise the else statement gets executed.

Example While loop with else

```
'''Example to illustrate
the use of else statement
with the while loop'''

counter = 0

while counter < 3:
    print("Inside loop")
    counter = counter + 1
else:
    print("Inside else")
```

Program Code

```
Inside loop
Inside loop
Inside loop
Inside else
```

Program Output

While loops can also have an optional else block.

The else part is executed if the condition in the while loop evaluates to False.

The while loop can be terminated with a break statement.

In such cases, the else part is ignored. Hence, a while loop's else part runs if no break occurs and the condition is false.

Here, we use a counter variable to print the string Inside loop three times.

On the fourth iteration, the condition in while becomes False. Hence, the else part is executed.

for Loop Statements

What is for loop in Python?

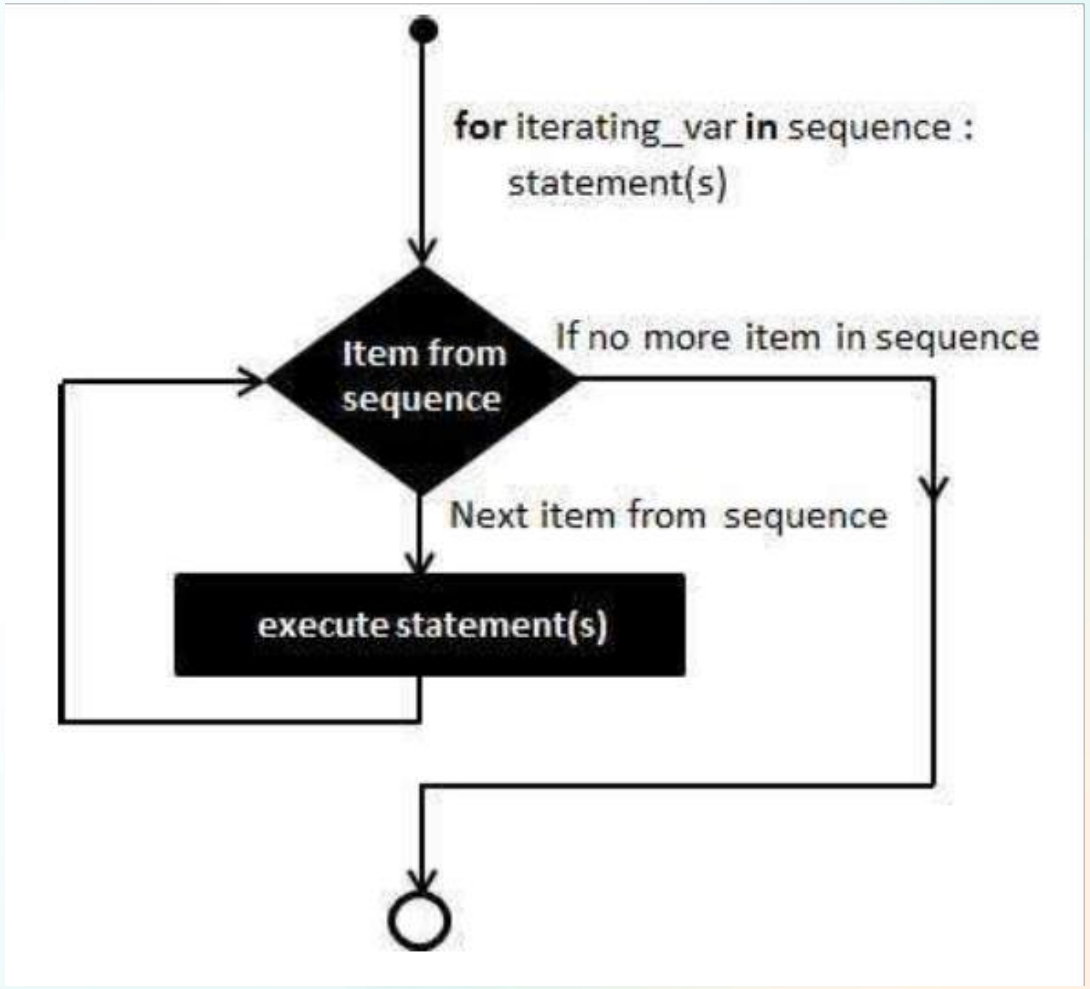
The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. Iterating over a sequence is called traversal.

Syntax and Flowchart of for Loop

```
for val in sequence:
        Body of for
```

Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.



Example: Python for Loop

```python
# Program to find the sum of all numbers stored in a list

# List of numbers
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]

# variable to store the sum
sum = 0

# iterate over the list
for val in numbers:
    sum = sum+val

print("The sum is", sum)
```

Program Code

```
The sum is 48
```

Program Output

The range() function

We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).

We can also define the start, stop and step size as range(start, stop,step_size). step_size defaults to 1 if not provided.

The range object is "lazy" in a sense because it doesn't generate every number that it "contains" when we create it. However, it is not an iterator since it supports in, len and __getitem__ operations.

This function does not store all the values in memory; it would be inefficient. So it remembers the start, stop, step size and generates the next number on the go.

To force this function to output all the items, we can use the function list().

Examples with range()

```
print(range(10))

print(list(range(10)))

print(list(range(2, 8)))

print(list(range(2, 20, 3)))
```

Program Code

```
range(0, 10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[2, 3, 4, 5, 6, 7]
[2, 5, 8, 11, 14, 17]
```

Program Output

Range in for loop

We can use the range() function in for loops to iterate through a sequence of numbers. It can be combined with the len() function to iterate through a sequence using indexing. Here is an example.

```
# Program to iterate through a list using indexing

genre = ['pop', 'rock', 'jazz']

# iterate over the list using index
for i in range(len(genre)):
    print("I like", genre[i])
```

Program Code

```
I like pop
I like rock
I like jazz
```

Program Output

Using else Statement with Loops

Python supports having an else statement associated with a loop statement.

- If the else statement is used with a for loop, the else block is executed only if for loops terminates normally (and not by encountering break statement).
- If the else statement is used with a while loop, the else statement is executed when the condition becomes false.

Example else statement with for Loop

```
numbers=[11,33,55,39,55,75,37,21,23,41,13]
for num in numbers:
    if num%2==0:
        print ('the list contains an even number')
        break
else:
    print ('the list doesnot contain even number')
```

Program Code

```
the list doesnot contain even number
```

Program Output

for loop with else

A for loop can have an optional else block as well.

The else part is executed if the items in the sequence used in for loop exhausts.

The break keyword can be used to stop a for loop. In such cases, the else part is ignored.
Hence, a for loop's else part runs if no break occurs.

Example else statement with for Loop

```
digits = [0, 1, 5]

for i in digits:
    print(i)
else:
    print("No items left.")
```

Program Code

```
0
1
5
No items left.
```

Program Output

Here, the for loop prints items of the list until the loop exhausts.

When the for loop exhausts, it executes the block of code in the else and prints No items left.

```python
# program to display student's marks from record
student_name = 'hadji'

marks = {'Joan': 90, 'Jeiven': 55, 'Johann': 77}

for student in marks:
    if student == student_name:
        print(marks[student])
        break
else:
    print('No entry with that name found.')
```

Program Code

```
No entry with that name found.
```

Program Output

This for...else statement can be used with the break keyword to run the else block only when the break keyword was not executed.

Nested loops

Python programming language allows the use of one loop inside another loop. The following section shows a few examples to illustrate the concept.

```
for iterating_var in sequence:
    for iterating_var in sequence:
        statements(s)
    statements(s)
```

```
while expression:
    while expression:
        statement(s)
    statement(s)
```

A final note on loop nesting is that you can put any type of loop inside any other type of loop.

For example a for loop can be inside a while loop or vice versa.

Example

```python
for i in range(1,11):
    for j in range(1,11):
        k=i*j
        print (k, end=' ')
    print()
```

Program Code

```
1 2 3 4 5 6 7 8 9 10
2 4 6 8 10 12 14 16 18 20
3 6 9 12 15 18 21 24 27 30
4 8 12 16 20 24 28 32 36 40
5 10 15 20 25 30 35 40 45 50
6 12 18 24 30 36 42 48 54 60
7 14 21 28 35 42 49 56 63 70
8 16 24 32 40 48 56 64 72 80
9 18 27 36 45 54 63 72 81 90
10 20 30 40 50 60 70 80 90 100
```

Program Output

The print() function inner loop has end=' ' which appends a space instead of default newline.

Hence, the numbers will appear in one row.

Last print() will be executed at the end of inner for loop.

Loop Control Statements

The Loop control statements change the execution from its normal sequence.

When the execution leaves a scope, all automatic objects that were created in that scope are destroyed.

Control Statement	Description
break statement	Terminates the loop statement and transfers execution to the statement immediately following the loop.
continue statement	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
pass statement	The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

Python break statement

The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.

If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.

Syntax and Flowchart

```
break
```

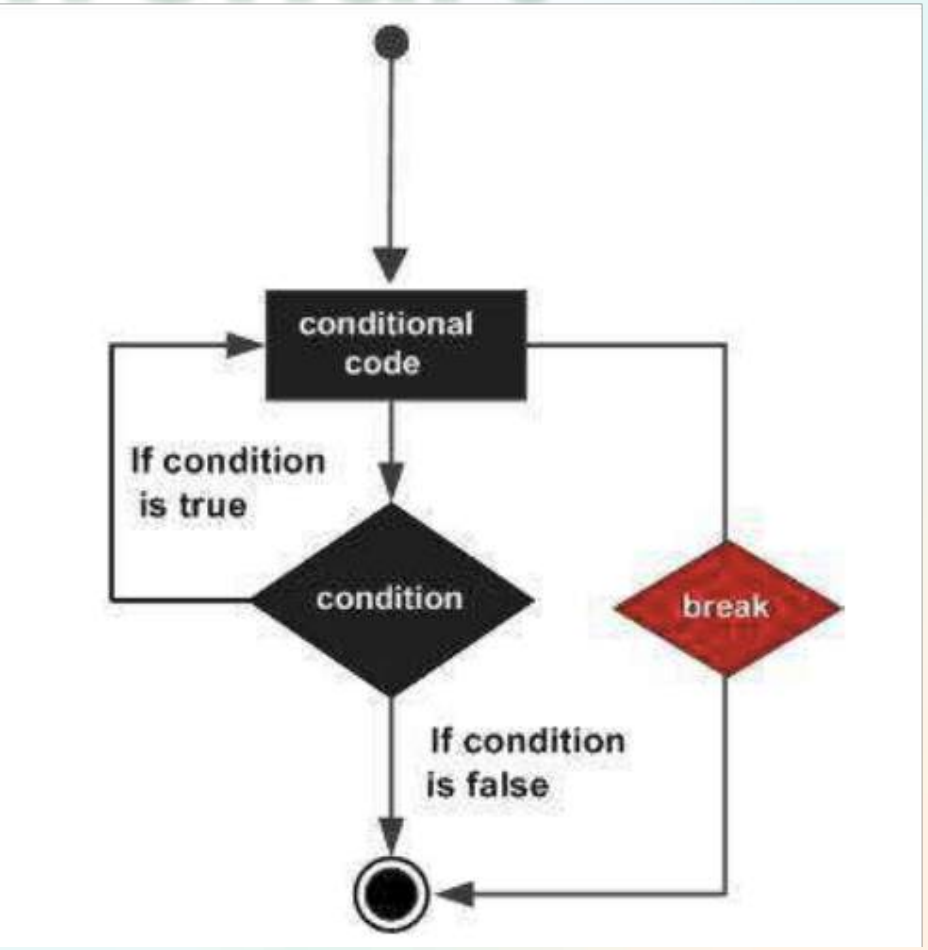
The working of break statement in for loop and while loop is shown below.

```
for var in sequence:
    # codes inside for loop
    if  condition:
        break
    # codes inside for loop

# codes outside for loop
```

```
while test expression:
    # codes inside while loop
    if  condition:
        break
    # codes inside while loop

# codes outside while loop
```



Example: Python break

```python
# Use of break statement inside the loop

for val in "string":
    if val == "i":
        break
    print(val)
print("The end")
```

Program Code

```
s
t
r
The end
```

Program Output

In this program, we iterate through the "string" sequence.

We check if the letter is i, upon which we break from the loop.

Hence, we see in our output that all the letters up till i gets printed. After that, the loop terminates.

Example: Python break

```python
for letter in 'Python': # First Example
    if letter == 'h':
        break
    print ('Current Letter :', letter)

var = 10 # Second Example
while var > 0:
        print ('Current variable value :', var)
        var = var -1
        if var == 5:
            break
print ("Goodbye!")
```

Program Code

```
Current Letter : P
Current Letter : y
Current Letter : t
Current variable value : 10
Current variable value : 9
Current variable value : 8
Current variable value : 7
Current variable value : 6
Goodbye!
```

Program Output

Example: Python break

```
no=int(input('any number: '))
numbers=[11,33,55,39,55,75,37,21,23,41,13]
for num in numbers:
    if num==no:
        print ('number found in list')
        break
else:
    print ('number not found in list')
```

Program Code

```
any number: 33
number found in list
```

Program Output

```
no=int(input('any number: '))
numbers=[11,33,55,39,55,75,37,21,23,41,13]
for num in numbers:
    if num==no:
        print ('number found in list')
        break
else:
    print ('number not found in list')
```

Program Code

```
any number: 5
number not found in list
```

Program Output

Python continue statement

The **continue** statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

Syntax/Flowchart

```
continue
```

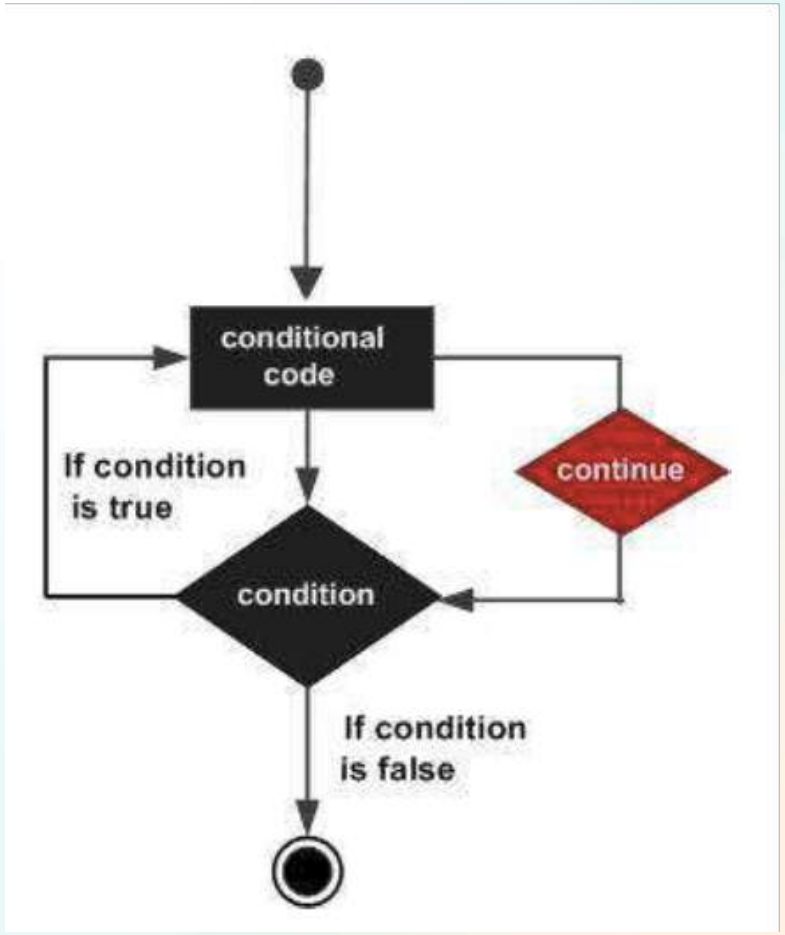
The working of continue statement in for and while loop is shown below.

```
for var in sequence:
    # codes inside for loop
    if  condition:
        continue
    # codes inside for loop

# codes outside for loop
```

```
while test expression:
    # codes inside while loop
    if  condition:
        continue
    # codes  inside while loop

# codes outside while loop
```



Example: Python continue

```python
# Program to show the use of continue statement inside loops

for val in "string":
    if val == "i":
        continue
    print(val)
print("The end")
```

Program Code

```
s
t
r
n
g
The end
```

Program Output

This program is same as the above example except the break statement has been replaced with continue.

We continue with the loop, if the string is i, not executing the rest of the block. Hence, we see in our output that all the letters except i gets printed.

Python pass statement

What is pass statement in Python?

In Python programming, the pass statement is a null statement.

The difference between a comment and a pass statement in Python is that while the interpreter ignores a comment entirely, pass is not ignored.

Syntax of pass

```
pass
```

Suppose we have a loop or a function that is not implemented yet, but we want to implement it in the future. They cannot have an empty body.

The interpreter would give an error. So, we use the pass statement to construct a body that does nothing.

Example: pass Statement

```python
for letter in 'Python':
    if letter == 'h':
        pass
        print ('This is pass block')
    print ('Current Letter :', letter)
print ("Good bye!")
```

Program Code

```
Current Letter : P
Current Letter : y
Current Letter : t
This is pass block
Current Letter : h
Current Letter : o
Current Letter : n
Good bye!
```

Program Output